

A FFT-based GMRES for fast solving of Poisson equation in concatenated geometry

Zichao Jiang^a, Zhuolin Wang^a, Jiacheng Lian^a, Qinghe Yao^{a,*}

^a*School of Aeronautics and Astronautics, Sun Yat-sen University, Shenzhen 518107, China*

Abstract

Fast Fourier Transform (FFT)-based solvers for the Poisson equation are highly efficient, exhibiting $O(N \log N)$ computational complexity and excellent parallelism. However, their application is typically restricted to simple, regular geometries due to the separability requirement of the underlying discrete operators. This paper introduces a novel domain decomposition method that extends the applicability of FFT-based solvers to complex composite domains—geometries constructed from multiple sub-regions. The method transforms the global problem into a system of sub-problems coupled through Schur complements at the interfaces. A key challenge is that the Schur complement disrupts the matrix structure required for direct FFT-based inversion. To overcome this, we develop a FFT-based preconditioner to accelerate the Generalized Minimal Residual (GMRES) method for the interface system. The central innovation is a novel preconditioner based on the inverse of the block operator without the Schur complement, which can be applied efficiently using the FFT-based solver. The resulting preconditioned iteration retains an optimal complexity for each step. Numerical experiments on a cross-shaped domain demonstrate that the proposed solver achieves the expected second-order accuracy of the underlying finite difference scheme. Furthermore, it exhibits significantly improved computational performance compared to a classic sparse GMRES solver based on Eigen library. For a problem with 10^5 grid points, our method achieves a speedup of over 40 times.

Keywords: FFT, Poisson solver, Concatenated geometry, Fast Fourier Transform (FFT), Domain decomposition method (DDM), low computational complexity

2010 MSC: 00-01, 99-00

*Corresponding author

Email address: yaoqhe@mail.sysu.edu.cn (Qinghe Yao)

1. Introduction

This paper focuses on the Helmholtz-Poisson equation of the form

$$\nabla^2 p + \kappa p = f, \mathbf{x} \in \Omega, \quad (1)$$

where $\mathbf{x} = (x, y, z)$ represents the spatial coordinates, and f is a given function on Ω . The equation presented in (1) can be transformed into various other equations, including the Schrödinger equation. When the constant $\kappa = 0$, the equation reduces to the Poisson equation, the solution of which holds significant importance for various applications, including incompressible flow simulations. Furthermore, the discrete scheme for this elliptic problem can be integrated into time-stepping schemes for parabolic and hyperbolic equations, such as the heat conduction and wave equations.

Over several decades, researchers have proposed numerous algorithms to enhance the computational efficiency of solving the equation (1). Among these, the most representative are algorithms based on the FFT[1, 2]. Originally developed for solving the Poisson equation on rectangular domains[3], these methods were subsequently widely applied to various large-scale problems, including turbulence simulation[4, 5]. Their widespread adoption is largely attributed to their computational complexity of $O(N \log N)$, where N denotes the total number of discrete grid points. Furthermore, these methods exhibit inherent parallelism, which makes them highly suitable for modern parallel computing architectures.

These methods typically employ discretizations based on the finite difference method (FDM) or the finite volume method (FVM)[6, 7, 8]. Initially, the global equations were transformed into independent sub-equations by expanding the solution in Fourier series, which were then solved using the Thomas algorithm. These approaches offer significant advantages in computational efficiency compared to Krylov subspace solvers and multigrid methods. However, these methods have a notable limitation. Their reliance on a spectral basis requires the discrete operator to be separable, a condition typically met only on structured, uniform grids. Consequently, these traditional methods makes the proposed methods challenging to handle problems with curved boundaries or complex geometries.

In recent years, to overcome the geometric adaptability limitations of traditional FFT algorithms, researchers have conducted extensive exploratory work. For instance, the FastRK3 solver [9] employs a discretization scheme on general curvilinear coordinates, as proposed by Nikitin [10]. Such methods are well-suited for one-dimensional curvilinear boundary problems. Similarly, Costa et al. [11, 12] adopted a comparable approach, leveraging FFT to transform three-dimensional problems into two-dimensional elliptic problems for solving high-resolution turbulence. However, the applicability of these solvers is restricted to problems with one-dimensional curvilinear boundaries, limiting their generalization to more complex geometries.

Another primary direction of investigation involves non-body-conforming grid methods, exemplified by the Immersed Boundary Method (IBM)[13]. Such methods construct interpolation schemes at the boundaries independently, thereby

44 preserving the structure of the underlying regular grid. This characteristic
 45 makes them highly suitable for integration with FFT-based solution algorithms,
 46 leading to extensive research and numerous applications[14, 15, 16, 17, 18]. Sub-
 47 sequent work has focused on improving both efficiency and accuracy. For in-
 48 stance, Lian et al. [19] introduced an overlapping computation strategy to
 49 enhance parallel efficiency, while Zhao et al. [20] proposed kernel-free inte-
 50 gration methods to reduce computational cost. In terms of accuracy, Ren et
 51 al. proposed a complex geometry handling method that combines the Matched
 52 Interface Boundary (MIB) technique with projection methods [21], and sub-
 53 sequently extended it to fourth-order precision[22], various types of boundary
 54 conditions [23] and multi-physics problems[24].

55 However, handling complex external boundary problems is notably challeng-
 56 ing for IBM-based methods. Moreover, the absence of body-fitted grids makes it
 57 difficult to apply local mesh refinement, which is often crucial for the resolution
 58 of boundary layers. These limitations collectively restrict the applicability of
 59 non-body-conforming grid methods [25].

60 In this paper, we propose a novel FFT-based domain decomposition algo-
 61 rithm that overcomes some of the crucial limitations of the aforementioned
 62 methods. To the best of our knowledge, this specific approach has not been
 63 previously explored in the published literature. The proposed method can be
 64 conceptually divided into two primary components: a domain decomposition
 65 strategy for the global equation and an FFT-based GMRES [26] algorithm for
 66 solving the resulting sub-equations. This algorithm relies on discrete equations
 67 expressed in matrix form. To ensure compatibility with this matrix formulation,
 68 an FFT-based solution algorithm, described through matrix eigenvalue trans-
 69 formations, is presented in this paper. In terms of computational performance,
 70 this algorithm exhibits the same $O(N \log N)$ complexity and inherent paral-
 71 lelism as solution algorithms based on spectral space expansion. Moreover, it
 72 can be more readily generalized to problems involving various types of boundary
 73 conditions.

74 In the domain decomposition algorithm proposed in this paper, the computa-
 75 tional domain Ω is decomposed into multiple sub-regions Ω_i ($i \in \mathbb{N}_+$) with sim-
 76 ple geometries, following the principles of DDMs. On each sub-region Ω_i , sub-
 77 equations suitable for FFT-based algorithms can be directly constructed. The
 78 connections between sub-regions are described by the Schur complement.[27]
 79 Similar to general DDMs, the global equation is thus transformed into sub-
 80 systems of equations on each sub-region.

81 The Schur complement disrupts the specific matrix structure required by
 82 FFT-based solvers, preventing a direct solution of the sub-equations. There-
 83 fore, this paper proposes an FFT-based preconditioned GMRES algorithm for
 84 solving these sub-equations. Specifically, the inverse of an approximate ma-
 85 trix that omits the Schur complement is employed as the preconditioner. This
 86 approach accelerates the Arnoldi process within GMRES by directly solving
 87 equations via FFT. While reducing the computational complexity of each it-
 88 eration to $O(N \log N)$, this method also significantly decreases the number of
 89 iterations. The overall computational complexity is then evaluated through

90 numerical results.

91 This paper is organized as follows. In Section 2, we begin by reviewing
 92 the FFT algorithm for simple geometries based on matrix eigenvalue transfor-
 93 mations, and then introduce the proposed FFT-based domain decomposition
 94 algorithm, including the FFT-preconditioned GMRES algorithm. To validate
 95 the solution efficiency of our proposed algorithm, Section 3 elaborates on the
 96 numerical results for the 2D Poisson equation within a cross-shape domain, in-
 97 cluding an analysis of the impact of parameters. Finally, Section 4 provides a
 98 summary of the methods proposed in this study.

99 2. Methodology

100 2.1. FFT-based Poisson algorithm

101 In this section, we begin by introducing the FFT-based Poisson equation
 102 solver algorithm for simple geometries, which forms the foundation of our pro-
 103 posed algorithm. Unlike traditional FFT-based solvers that rely on spectral
 104 expansion, the FFT-based solver for simple geometries proposed in this paper
 105 primarily focuses on low-complexity feature transformations of matrix equa-
 106 tions, including the implementation of distinct FFT-based Poisson equation
 107 solvers for various boundary conditions.

108 The two-dimensional Poisson equation $\nabla^2 u = f$ in $\Omega \subset \mathbb{R}^2$ is discretized by
 109 a central difference scheme, such that

$$\frac{p_{i+1,j} - 2p_{i,j} + p_{i-1,j}}{\Delta_x^2} + \frac{p_{i,j+1} - 2p_{i,j} + p_{i,j+1}}{\Delta_y^2} = f_{i,j}, \quad (2)$$

110 where Δ_x and Δ_y are the spatial steps on x and y direction, respectively.

111 Traditional FFT-based Poisson equation solvers are typically applicable only
 112 when Ω is a simple geometry, such as a rectangle in a Cartesian coordinate
 113 system or an annulus in a polar coordinate system. To simplify the presentation,
 114 we describe the discretization of the Poisson equation on a two-dimensional
 115 rectangular domain Ω in a Cartesian coordinate system. A uniform mesh is
 116 employed, with the number of nodes in the x and y directions being $(m+2)$
 117 and $(n+2)$ respectively, including the boundary nodes. The central difference
 118 scheme (2) for the interior points can thus be written as the $mn \times mn$ block
 119 tridiagonal matrix equation

$$\begin{bmatrix} \mathbf{A}_y & \delta_x \mathbf{I} & & \\ \delta_x \mathbf{I} & \mathbf{A}_y & \ddots & \\ & \ddots & \ddots & \delta_x \mathbf{I} \\ & & \delta_x \mathbf{I} & \mathbf{A}_y \end{bmatrix} \begin{bmatrix} \mathbf{p}_1 \\ \mathbf{p}_2 \\ \vdots \\ \mathbf{p}_m \end{bmatrix} = \begin{bmatrix} \mathbf{f}_1 \\ \mathbf{f}_2 \\ \vdots \\ \mathbf{f}_m \end{bmatrix}. \quad (3)$$

120 In (3), $\delta_x = \Delta_x^{-2}$ and $\delta_y = \Delta_y^{-2}$. The vector $\mathbf{p} = [p_{i,1} \ p_{i,2} \ \cdots \ p_{i,n}]^T$ con-
 121 tains the unknown values along the i -th grid line, and the vector $\mathbf{f}$ is defined

analogously. Assuming that the Poisson equation has Dirichlet boundary conditions $p|_{\partial\Omega} = 0$ on all four boundaries of the rectangular domain, the $n \times n$ submatrix $\mathbf{A}_y$ is

$$\mathbf{A}_y = \mathbf{A}_y^D = \begin{bmatrix} -2(\delta_x + \delta_y) & \delta_y & & \\ \delta_y & -2(\delta_x + \delta_y) & \ddots & \\ & \ddots & \ddots & \delta_y \\ & & \delta_y & -2(\delta_x + \delta_y) \end{bmatrix}. \quad (4)$$

The matrix $\mathbf{A}_y^D$ is a symmetric Toeplitz matrix whose eigenvectors form the basis of the Discrete Sine Transform (DST). This property is exploited by traditional FFT-based solvers. The solution is expanded in the basis of these eigenvectors along the y -direction:

$$p_{i,j} = \sum_{k=1}^n \hat{p}_{i,k} \sin\left(\frac{jk\pi}{n+1}\right), \quad (5)$$

where $\hat{p}_{i,k}$ are the spectral coefficients. Substituting this expansion into the finite difference scheme decouples the system of equations. For each spectral mode k , we obtain an independent tridiagonal system for the coefficients $\hat{p}_{i,k}$ along the x -direction:

$$\delta_x \hat{p}_{i-1,k} + \lambda_k \hat{p}_{i,k} + \delta_x \hat{p}_{i+1,k} = \hat{f}_{i,k}, \quad (6)$$

where $\hat{f}_{i,k}$ are the transformed source terms and the k -th eigenvalue λ_k of the submatrix $\mathbf{A}_y^D$ is defined by

$$\lambda_k = -2(\delta_x + \delta_y) + 2\delta_y \cos\left(\frac{k\pi}{n+1}\right). \quad (7)$$

It can be observed that these equations are independent, allowing the global equation to be transformed into multiple independent sub-equations, which is particularly suitable for parallel computation. Furthermore, each transformed sub-equation (6) possesses a tridiagonal structure, which can be efficiently solved using the Thomas algorithm with a complexity of $O(m)$. Moreover, (5) can be directly computed with $O(n \log n)$ complexity using the discrete sine transform accelerated by FFT. Therefore, FFT-based solvers have become one of the fastest algorithms for solving Poisson equations in simple geometries.

However, traditional FFT-based Poisson equation solvers relying on spectral expansion face challenges posed by potential aliasing issues. Additionally, for various boundary conditions, the process of constructing basis functions can be complex, or it may be difficult to devise suitable spectral expansion methods. Therefore, the primary idea behind the FFT-based Poisson equation solver algorithm adopted in this study is the eigenvalue decomposition of (3). The FFT algorithm is then utilized to accelerate this transformation process. This algorithm shares similar properties with spectral expansion methods, specifically its

151 ability to transform a global equation into independent sub-equations, thereby
 152 making it suitable for parallel computing.

153 This section details the FFT solver based on eigendecomposition, using
 154 Dirichlet boundary conditions $p|_{\partial\Omega} = 0$ as an illustrative example. The eigen-
 155 values λ_j^D of matrix $\mathbf{A}_y^D$ are same as (7). Let the j -th eigenvector be $\mathbf{q}_j^D$, whose
 156 elements are

$$q_{j,k}^D = \sqrt{\frac{2}{n+1}} \sin \frac{jk\pi}{n+1}, \quad \text{for } k = 1, \dots, n. \quad (8)$$

157 Define $\mathbf{Q}_D$ as the matrix whose columns are the eigenvectors $\mathbf{q}_j^D$. Consequently,
 158 it can be further shown that $\mathbf{\Lambda}_y^D = \mathbf{Q}_D \mathbf{A}_y^D \mathbf{Q}_D^T$, where $\mathbf{\Lambda}_y$ is a diagonal matrix
 159 with λ_j^D as its diagonal elements. By applying an eigenvalue transformation,
 160 equation (3) can be transformed into

$$\begin{bmatrix} \mathbf{\Lambda}_y & \delta_x \mathbf{I} & & \\ \delta_x \mathbf{I} & \mathbf{\Lambda}_y & \ddots & \\ & \ddots & \ddots & \delta_x \mathbf{I} \\ & & \delta_x \mathbf{I} & \mathbf{\Lambda}_y \end{bmatrix} \begin{bmatrix} \hat{\mathbf{p}}_1 \\ \hat{\mathbf{p}}_2 \\ \vdots \\ \hat{\mathbf{p}}_n \end{bmatrix} = \begin{bmatrix} \hat{\mathbf{f}}_1 \\ \hat{\mathbf{f}}_2 \\ \vdots \\ \hat{\mathbf{f}}_n \end{bmatrix}, \quad (9)$$

161 where the vector $\hat{\mathbf{p}}_i = \mathbf{Q}^T \mathbf{p}_i$ and $\hat{\mathbf{f}}_i = \mathbf{Q}^T \mathbf{f}_i$.

162 $\hat{\mathbf{p}}'_j$ is defined by reordering elements of $\hat{\mathbf{p}}_j$, such that

$$\hat{\mathbf{p}}'_j = [\hat{u}_{1,j} \quad \hat{u}_{2,j} \quad \dots \quad \hat{u}_{n,j}]^T, \quad (10)$$

163 and the vector $\hat{\mathbf{f}}'_j$ is defined analogously. (9) can be further rearranged into

$$\begin{bmatrix} \hat{\mathbf{A}}_1 & & & \\ & \hat{\mathbf{A}}_2 & & \\ & & \ddots & \\ & & & \hat{\mathbf{A}}_m \end{bmatrix} \begin{bmatrix} \hat{\mathbf{p}}'_1 \\ \hat{\mathbf{p}}'_2 \\ \vdots \\ \hat{\mathbf{p}}'_m \end{bmatrix} = \begin{bmatrix} \hat{\mathbf{f}}'_1 \\ \hat{\mathbf{f}}'_2 \\ \vdots \\ \hat{\mathbf{f}}'_m \end{bmatrix}, \quad (11)$$

164 where $\hat{\mathbf{A}}_k$ is the tridiagonal matrix forms that

$$\hat{\mathbf{A}}_k = \begin{bmatrix} \lambda_k^D & \delta_x & & \\ \delta_x & \lambda_k^D & \ddots & \\ & \ddots & \ddots & \delta_x \\ & & \delta_x & \lambda_k^D \end{bmatrix}. \quad (12)$$

165 (11) can be directly transformed into n independent sub-equations, $\hat{\mathbf{A}}_k \hat{\mathbf{p}}'_k = \hat{\mathbf{f}}'_k$.
 166 Each of these sub-equations possesses a tridiagonal matrix on its left-hand side,
 167 as shown in (12). This indicates that the FFT-based Poisson equation solver
 168 algorithm, when derived through eigenvalue transformation, shares the same
 169 characteristics as those obtained via spectral expansion.

170 The foundation of the FFT-based Poisson equation solver algorithm em-
 171 ployed in this paper lies in the fast computation of $\mathbf{Q}_D \mathbf{p}$ and $\mathbf{Q}_D^T \mathbf{p}$ for an arbi-
 172 trary vector $\mathbf{p}$ of length n , achieved with $O(n \log n)$ complexity using the FFT
 173 algorithm. The core of this algorithm is to achieve the rapid computation of
 174 $\mathbf{Q}_D \mathbf{p}$ and $\mathbf{Q}_D^T \mathbf{p}$ through a combination of the FFT algorithm and other low-
 175 complexity operations. It is straightforward to demonstrate that the DFT of a
 176 vector $\mathbf{p}$ is equivalent to the linear transformation

$$\mathcal{F}[\mathbf{p}] = \mathbf{D}_n \mathbf{p}, \quad (13)$$

177 where $\mathbf{D}_n$ is a symmetric matrix of size n , and its (j, k) element is given by

$$d_{j,k} = e^{\frac{-2\pi i(j-1)(k-1)}{n}}, \quad (14)$$

178 where i is the imaginary unit. The matrix $\mathbf{D}_n$ can be further decomposed into
 179 $\mathbf{D}_n = \mathbf{C}_n + i\mathbf{S}_n$, where $\mathbf{C}_n$ and $\mathbf{S}_n$ represent its real and imaginary parts,
 180 respectively. It can be further shown that

$$\mathbf{Q}_D = \mathbf{K}_D \mathbf{S}_{2n}, \quad (15)$$

181 where $\mathbf{K}_D$ is an $n \times 2n$ matrix with elements of 1 at each $(j, 2j)$ and zeros
 182 elsewhere. Therefore $\mathbf{Q}_D \mathbf{p}_i$ can be computed with a complexity of $O(n \log n)$
 183 using an FFT algorithm.

184 Although this section only presents the FFT-based method for 2D problems,
 185 the application of this method can be expanded to 3D cases. For 3D problems,
 186 another transformation is necessary to transform the 3D global equation into
 187 the independent 2D subequations.

188 2.2. Boundary conditions

189 The FFT-based solver algorithm presented previously, when applied to prob-
 190 lems with Neumann and periodic boundary conditions, differs from the Dirichlet
 191 case only in the specific form of the transformation matrix $\mathbf{Q}$ and the corre-
 192 sponding fast transform algorithm.. This section introduces the low-complexity
 193 transform algorithms for these two types of boundaries, respectively.

194 For Neumann boundary conditions, where $\frac{\partial p}{\partial y} = 0$ on the boundaries in the
 195 y -direction, the submatrix $\mathbf{A}_y$ forms

$$\mathbf{A}_y = \mathbf{A}_y^N = \begin{bmatrix} -2\delta_x - \delta_y & \delta_y & & & \\ \delta_y & -2(\delta_x + \delta_y) & \ddots & & \\ & \ddots & \ddots & \delta_y & \\ & & \delta_y & -2\delta_x - \delta_y & \end{bmatrix}. \quad (16)$$

196 The eigenvalues λ_j^N can be calculated as

$$\lambda_j^N = -2(\delta_x + \delta_y) + 2\delta_y \cos \frac{(j-1)\pi}{n-1}, \quad \text{for } j = 1, \dots, n. \quad (17)$$

197 The matrix $\mathbf{Q}_N$, containing the eigenvectors of $\mathbf{A}_y^N$, is given by

$$\mathbf{Q}_N = \bar{\mathbf{Q}}_N + \frac{1 + \sqrt{2}}{\sqrt{n}} \mathbf{E}_n, \quad (18)$$

198 where $\mathbf{E}_n$ is the matrix where only the n -th column contains ones, with all other
199 columns being zeros. The elements of matrix $\bar{\mathbf{Q}}_N$ are given by

$$\bar{q}_{j,k}^N = \sqrt{\frac{2}{n}} \sin \frac{(jk - k - 2nj) \pi}{2n} \quad (19)$$

200 Therefore, $\bar{\mathbf{Q}}_N$ can be further decomposed as

$$\bar{\mathbf{Q}}_N = \sqrt{\frac{2}{n}} \mathbf{K}_N (\mathbf{S}_n \mathbf{\Delta}_C + \mathbf{C}_n \mathbf{\Delta}_S), \quad (20)$$

201 where matrices $\mathbf{K}_N$, $\mathbf{\Delta}_C$, and $\mathbf{\Delta}_S$ are all $n \times n$ diagonal matrices, with their k -th
202 elements being $(-1)^k$, $\cos \frac{k\pi}{2n}$, and $\sin \frac{k\pi}{2n}$, respectively. Similar to the Dirich-
203 let boundary conditions, when computing $\mathbf{Q}_N \mathbf{p}$, the computational process can
204 be progressively decomposed according to (18) and (20). Since the left mul-
205 tiplication of vectors by matrices $\mathbf{C}_n$ and $\mathbf{S}_n$ can be calculated via FFT, and
206 all other involved matrices are diagonal, the computational process, similar to
207 the Dirichlet boundary case, can be transformed into one with a complexity of
208 $O(n \log n)$.

209 It is worth noting that $\bar{\mathbf{Q}}_N$ is not symmetric. Therefore, unlike the Dirich-
210 let boundary conditions, when computing $\mathbf{Q}_N^T \mathbf{p}$, it is necessary to rely on the
211 decomposition of $\mathbf{Q}_N^T$, i.e.,

$$\mathbf{Q}_N^T = \frac{1 + \sqrt{2}}{\sqrt{n}} \mathbf{E}_n^T + \sqrt{\frac{2}{n}} (\mathbf{\Delta}_C \mathbf{S}_n + \mathbf{\Delta}_S \mathbf{C}_n) \mathbf{K}_N. \quad (21)$$

212 This yields a computational process different from that for $\mathbf{Q}_N \mathbf{p}$. However, it
213 can be readily shown that this process also has a computational complexity of
214 $O(n \log n)$.

215 In traditional spectral expansion methods, periodic boundary conditions typ-
216 ically the simplest to implement. However, for the algorithm proposed in this
217 section, the matrix corresponding to periodic boundary conditions is more com-
218 plex than the matrix decompositions for Dirichlet problems (15) and Neumann
219 boundary problems (20). The matrix $\mathbf{A}_y$ corresponding to periodic boundary
220 conditions is given by

$$\mathbf{A}_y = \mathbf{A}_y^P = \begin{bmatrix} -2(\delta_x + \delta_y) & \delta_y & & \delta_y \\ \delta_y & -2(\delta_x + \delta_y) & \ddots & \\ & \ddots & \ddots & \delta_y \\ \delta_y & & \delta_y & -2(\delta_x + \delta_y) \end{bmatrix}. \quad (22)$$

221 The eigenvalues λ_j^P can be calculated as

$$\lambda_j^P = -2(\delta_x + \delta_y) + 2\delta_y \cos \frac{2\pi(j-1)}{n}, \quad \text{for } j = 1, \dots, n. \quad (23)$$

222 The elements of the matrix $\mathbf{Q}_P$, which is composed of the eigenvectors of $\mathbf{A}_y^P$,
 223 are given by

$$q_{j,k}^P = \sqrt{\frac{2}{n}} \sin \left[\frac{2j\pi}{n} \left\lfloor \frac{n+1-j}{2} \right\rfloor + \frac{\pi(2k-1)}{4} \right] \quad (24)$$

224 where $\lfloor \cdot \rfloor$ denotes the floor function. Therefore, $\mathbf{Q}_P$ can be decomposed as

$$\mathbf{Q}_P = \sqrt{\frac{2}{n}} \left(\mathbf{S}^P \mathbf{\Delta}_{PC} + \mathbf{C}^P \mathbf{\Delta}_{PS} \right), \quad (25)$$

225 where matrices $\mathbf{\Delta}_{PC}$ and $\mathbf{\Delta}_{PS}$ are diagonal matrices whose k -th diagonal ele-
 226 ments are $\cos \frac{(2k-1)\pi}{4}$ and $\sin \frac{(2k-1)\pi}{4}$, respectively. The decomposition in Eq.
 227 (25) holds if and only if the product n is an even number. To satisfy this con-
 228 dition, we assume throughout this work that n is an even integer. Accordingly,
 229 the elements of matrix $\mathbf{S}^P$ are

$$s_{j,k}^P = \begin{cases} \sin \frac{j\pi(n-2l)}{n}, & l \in \mathbb{N}, k = 2l \\ \sin \frac{j\pi(n-2l+2)}{n}. & l \in \mathbb{N}, k = 2l+1 \end{cases} \quad (26)$$

230 Similarly, the elements of $\mathbf{C}^P$ are

$$c_{j,k}^P = \begin{cases} \cos \frac{j\pi(n-2l)}{n}, & l \in \mathbb{N}, k = 2l \\ \cos \frac{j\pi(n-2l+2)}{n}. & l \in \mathbb{N}, k = 2l+1 \end{cases} \quad (27)$$

231 To facilitate an FFT-based implementation, we partition the matrices $\mathbf{C}^P$ and
 232 $\mathbf{S}^P$ based on column parity into four intermediate $n/2 \times n$ matrices. The matrix
 233 $\mathbf{C}^{PE}$ is constructed from the even-indexed columns of $\mathbf{C}^P$, and $\mathbf{C}^{PO}$ from the
 234 odd-indexed columns. Their elements are given by

$$c_{j,l}^{PE} = \cos \frac{j\pi(n-2l)}{n} = (-1)^j \cos \frac{2\pi jl}{n}, \quad (28)$$

235 and

$$c_{j,l}^{PO} = \cos \frac{j\pi(n-2l+2)}{n}. \quad (29)$$

236 The matrices $\mathbf{S}^{PE}$ and $\mathbf{S}^{PO}$ are constructed analogously from the even and
 237 odd columns of $\mathbf{S}^P$, respectively. It can be shown that $\mathbf{C}^{PE}$ and $\mathbf{S}^{PE}$ correspond
 238 to the upper halves of $\mathbf{K}_N \mathbf{S}_{2n}$ and $\mathbf{K}_N \mathbf{S}_{2n}$, whose low-complexity algorithm has
 239 been introduced above. Consequently, the matrix-vector product $\mathbf{Q}_P \mathbf{p}$ can be
 240 efficiently computed using two FFTs, following a strategy analogous to that
 241 employed for the Neumann problem.

242 In this section, we have detailed an FFT-based algorithm derived from the
 243 eigendecomposition of the discrete operator. A notable feature of this method

244 is that, contrary to classic FFT-based algorithm where periodic problems are
 245 simplest. Nevertheless, we have established a low-complexity, FFT-based solver
 246 for each of the three boundary condition types (Dirichlet, Neumann, and pe-
 247 riodic). This unified framework is fundamental to the domain decomposition
 248 algorithm presented in the subsequent sections.

249 2.3. Domain decomposition framework

250 This section introduces the proposed DDM, using the two-dimensional L-
 251 shaped region shown in Fig.1 as an example. The two-dimensional L-shaped
 252 region in Fig.1 can be decomposed into three connected rectangular sub-regions,
 253 labeled as Ω_1 , Ω_2 , and Ω_3 , respectively.

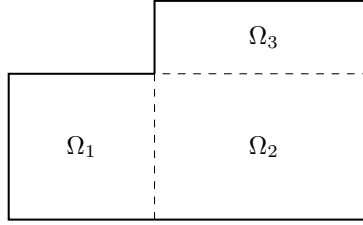


Figure 1: The schematic of the L-shaped region Ω with subdomains Ω_1 , Ω_2 , and Ω_3 .

254 The global equation has the block matrix structure, which forms

$$\begin{bmatrix} \mathbf{A}_1 & \mathbf{R}_{12} & \\ \mathbf{R}_{21} & \mathbf{A}_2 & \mathbf{R}_{23} \\ & \mathbf{R}_{32} & \mathbf{A}_3 \end{bmatrix} \begin{bmatrix} \mathbf{p}_1 \\ \mathbf{p}_2 \\ \mathbf{p}_3 \end{bmatrix} = \begin{bmatrix} \mathbf{f}_1 \\ \mathbf{f}_2 \\ \mathbf{f}_3 \end{bmatrix}, \quad (30)$$

255 where the sparse matrices $\mathbf{R}_{ij}$ represent the coupling across the interface be-
 256 tween subdomains Ω_i and Ω_j . For the self-adjoint Poisson operator, these matri-
 257 ces satisfy $\mathbf{R}_{ij} = \mathbf{R}_{ji}^T$. Each sub-equation $\mathbf{A}_i \mathbf{q}_i = \mathbf{f}_i$ ($i = 1, 2, 3$) can be directly
 258 interpreted as a Poisson equation within the sub-domain Ω_i , where the external
 259 boundary conditions are consistent with those of the original domain Ω , and
 260 the boundary conditions at the interfaces between sub-domains are treated as
 261 Dirichlet boundary conditions.

262 Therefore, each sub-equation possesses at least one Dirichlet boundary, which
 263 implies that $\mathbf{A}_1$, $\mathbf{A}_2$, and $\mathbf{A}_3$ are all invertible matrices. Consequently, by ap-
 264 plying the Schur complement, the global system (30) can be transformed into
 265 three sub-equations:

$$\mathbf{p}_1 = \mathbf{q}_1 - \mathbf{A}_1^{-1} \mathbf{R}_{12} \mathbf{p}_2, \quad (31a)$$

$$(\mathbf{A}_2 - \mathbf{S}_{21} - \mathbf{S}_{23}) \mathbf{p}_2 = \mathbf{f}_2 - \mathbf{R}_{21} \mathbf{q}_1 - \mathbf{R}_{23} \mathbf{q}_3, \quad (31b)$$

$$\mathbf{p}_3 = \mathbf{q}_3 - \mathbf{A}_3^{-1} \mathbf{R}_{32} \mathbf{p}_2, \quad (31c)$$

266 where $\mathbf{S}_{ij} = \mathbf{R}_{ij} \mathbf{A}_i^{-1} \mathbf{R}_{ji}$ and $\mathbf{q}_i = \mathbf{A}_i^{-1} \mathbf{f}_i$.

Since each subdomain is rectangular, the matrices $\mathbf{A}_1$, $\mathbf{A}_2$, and $\mathbf{A}_3$ all possess the block-tridiagonal structure shown in (3), which is required for the FFT-based solver. Consequently, the equations $\mathbf{q}_i = \mathbf{A}_i^{-1} \mathbf{f}_i$ can be solved efficiently using the algorithm described in Section 2.1. However, it is evident that the structure of the left-hand side matrix $\mathbf{L}_2 = \mathbf{A}_2 - \mathbf{S}_{21} - \mathbf{S}_{23}$ in (31b) does not satisfy the conditions for direct solution using FFT-based algorithms. Therefore, an iterative solution algorithm is proposed in this paper.

2.4. FFT-based GMRES algorithm

Let equation (31b) be simplified as $\mathbf{L}_2 \mathbf{p}_2 = \mathbf{f}'_2$, where $\mathbf{f}'_2 = \mathbf{f}_2 - \mathbf{R}_{21} \mathbf{q}_1 - \mathbf{R}_{23} \mathbf{q}_3$. Due to the sparsity of $\mathbf{S}_{21}$ and $\mathbf{S}_{23}$, a fixed-point iteration can be directly formulated as

$$\mathbf{p}^{(k+1)} = \mathbf{A}_2^{-1} (\mathbf{S}_{21} + \mathbf{S}_{23}) \mathbf{p}^{(k)} + \mathbf{q}'_2, \quad (32)$$

where $\mathbf{p}^{(k)}$ represents the solution at the k -th iteration step. In this iterative process, each iteration step effectively involves solving the equation

$$\mathbf{A}_2 \mathbf{p}^{(k+1)} = \mathbf{S}_{21} + \mathbf{S}_{23} \mathbf{p}^{(k)} + \mathbf{f}'_2. \quad (33)$$

This equation can be directly solved using the algorithm described in Section 2.1, which theoretically yields an $O(n \log n)$ complexity for each iteration step. However, a critical issue is that the matrix $\mathbf{A}_2$ is not diagonally dominant. Consequently, the convergence of this simple fixed-point iteration is not guaranteed. To address this problem, an improved Generalized Minimal Residual (GMRES) algorithm, enhanced by FFT, is proposed in this paper to efficiently solve (31b) based on the matrix structure.

Since $\mathbf{L}_2 = (\mathbf{A}_2 - \mathbf{S}_{21} - \mathbf{S}_{23})$ is generally not symmetric, algorithms like Conjugate Gradient (CG), which require specific matrix structures, are not applicable. GMRES is therefore one of the most widely employed choices, balancing both convergence efficiency and stability. The computational process of the GMRES algorithm can be decomposed into the Arnoldi process and a least-squares problem on the Krylov subspace. Let the dimension of the Krylov subspace be m . Assuming the initial residual of the iteration is $\mathbf{r}^{(0)}$, the iterative process of the GMRES algorithm is as follows:

1. Generate the Krylov subspace basis vectors via $\mathbf{w}_i = \mathbf{L}_2 \mathbf{w}_{i-1}$ ($i = 2, \dots, m$), where $\mathbf{w}_1 = \mathbf{r}^{(0)}$.
2. Orthogonalize the Krylov subspace basis vectors $\mathbf{w}_i$ using the Gram-Schmidt method and achieve $V_d = [\mathbf{v}_1, \dots, \mathbf{v}_d]$.
3. Solve the least-squares problem $y_d^* = \arg \min_{y_d \in \mathbb{R}^d} \|\|\mathbf{r}^{(0)}\|_2 \mathbf{e}_1 - \bar{H}_d \mathbf{y}_d\|_2$ on the constructed Krylov space, where $\bar{H}_d$ is the Hessenberg matrix obtained from the Arnoldi process.
4. Update the approximate solution: Calculate $\mathbf{p}^{(k+1)} = \mathbf{p}^{(k)} + V_d \mathbf{y}_d^*$.
5. If the residual $\mathbf{r}^{(k)}$ falls below a preset tolerance, the iteration terminates; otherwise, the process continues.

305 It is straightforward to demonstrate that for general problems, the dimen-
 306 sion of the Krylov subspace, m , is typically chosen to be much smaller than the
 307 number of unknowns in the subdomain, n_2 . Consequently, the Arnoldi process
 308 often constitutes the most time-consuming part of the GMRES solver. There-
 309 fore, the core idea of the optimized GMRES algorithm proposed in this paper is
 310 to use $\mathbf{A}_2$ as a preconditioner is to employ $\mathbf{A}_2^{-1}$ as a right preconditioner. This
 311 transforms the system and accelerates convergence, while the FFT-based solver
 312 for $\mathbf{A}_2$ allows the preconditioning step to be performed efficiently.

313 Specifically, in the proposed method, the equation $\mathbf{L}_2 \mathbf{p}_2 = \mathbf{f}'_2$ is transformed
 314 into

$$\mathbf{p}_2 - \mathbf{A}_2^{-1} (\mathbf{S}_{21} + \mathbf{S}_{23}) \mathbf{p}_2 = \mathbf{A}_2^{-1} \mathbf{f}'_2. \quad (34)$$

315 The generation of Krylov subspace basis vectors, originally calculated by $\mathbf{w}_i =$
 316 $\mathbf{L}_2 \mathbf{w}_{i-1}$, is replaced by

$$\mathbf{w}_i = \mathbf{w}_{i-1} - \mathbf{w}'_{i-1} \quad (35)$$

317 where $\mathbf{w}'_{i-1}$ is obtained by solving the equation

$$\mathbf{A}_2 \mathbf{w}'_{i-1} = (\mathbf{S}_{21} + \mathbf{S}_{23}) \mathbf{w}_{i-1} \quad (36)$$

318 using the FFT-based algorithm introduced in 2.1.

319 In summary, the proposed FFT-based domain decomposition algorithm for
 320 solving the global system in Eq. (30) proceeds as 1.

Algorithm 1: FFT-based DDM algorithm

- 1 Solve $\mathbf{A}_1 \mathbf{q}_1 = \mathbf{f}_1$ for $\mathbf{q}_1$ using the FFT-based solver.
 - 2 Solve $\mathbf{A}_3 \mathbf{q}_3 = \mathbf{f}_3$ for $\mathbf{q}_3$ using the FFT-based solver.
 - 3 Form the modified right-hand side $\mathbf{f}'_2 = \mathbf{f}_2 - \mathbf{R}_{21} \mathbf{q}_1 - \mathbf{R}_{23} \mathbf{q}_3$.
 - 321 4 Solve the preconditioned system from Eq. (34) for $\mathbf{p}_2$ using the
 FFT-based GMRES algorithm.
 - 5 Form the right-hand sides $\mathbf{f}'_1 = \mathbf{f}_1 - \mathbf{R}_{12} \mathbf{p}_2$ and $\mathbf{f}'_3 = \mathbf{f}_3 - \mathbf{R}_{32} \mathbf{p}_2$.
 - 6 Solve $\mathbf{A}_1 \mathbf{p}_1 = \mathbf{f}'_1$ for $\mathbf{p}_1$ using the FFT-based solver.
 - 7 Solve $\mathbf{A}_3 \mathbf{p}_3 = \mathbf{f}'_3$ for $\mathbf{p}_3$ using the FFT-based solver.
-

322 This constitutes the general procedure for the FFT-based domain decom-
 323 position algorithm proposed in this paper. This process can also be directly
 324 extended to solve Poisson equations within more complex concatenated geome-
 325 tries than that shown in Figure 1. The computational procedure of the entire
 326 domain decomposition algorithm categorizes the sub-domains into two types:
 327 As exemplified by sub-domains Ω_1 and Ω_3 in Figure 1, those sub-domains that
 328 are connected to only one other sub-domain can be directly solved using the
 329 FFT-based algorithm introduced in Section 2.1; these are thus termed inde-
 330 pendent sub-domains. Conversely, sub-domains like Ω_2 , which are adjacent to
 331 multiple other sub-domains, require their sub-equations to be solved using the
 332 proposed GMRES algorithm; these are referred to as coupled sub-domains. By
 333 employing the two distinct algorithms proposed in this paper, fast solutions
 334 with $O(n \log n)$ complexity are achieved for both types of sub-domains. It is
 335 worth noting that the subspace dimension m plays a crucial role in determining

the convergence rate of the iteration. Consequently, the overall computational complexity of the solution algorithm is jointly determined by the equation size and the subspace dimension m . This paper provides a detailed evaluation of the overall complexity of the solution algorithm based on numerical results in Section 3.3.

3. Numerical results

In this research, all programs used for algorithm evaluation are in-house codes, developed in C++ and utilizing the highly optimized FFTW library for all FFT computations [28, 29]. All source code, including the specific versions employed in this research, is open-source and publicly available on https://github.com/ffskibkwi/Focalors_Poisson.

Our solver provides two distinct parallel implementations: one based on OpenMP for shared-memory environments, and another based on MPI for distributed-memory systems. Both parallel versions are made available in the aforementioned open-source repository. For consistency and relevance to single-node performance, all computational performance results, including execution times and scalability metrics presented in this study, are exclusively derived from the OpenMP-parallelized version of the code.

All computations were executed on a single compute node, configured with two 26-core Intel Xeon CPUs (total 52 cores) operating at a base frequency of 2.1 GHz, and equipped with 256 GB of node memory.

3.1. Precision validation

In this study, to evaluate the reliability and computational efficiency of the proposed Poisson equation solver, we construct a test case on a two-dimensional cross-shaped domain, as depicted in Figure 2.

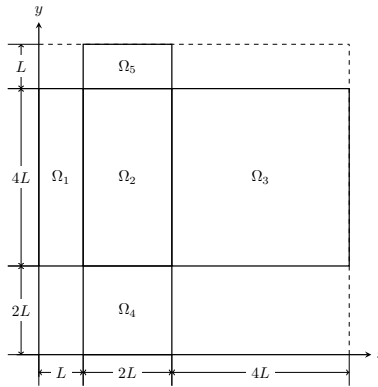


Figure 2: Cross-shape geometry.

361 In Figure 2, we defined the scale factor as $L = 1/7$, and the equations were
 362 set with the following boundary conditions:

$$\begin{cases} \frac{\partial^2 p}{\partial x^2} + \frac{\partial^2 p}{\partial y^2} = f(x, y), \\ p|_{x=0} = p|_{y=0} = p|_{x=7L} = p|_{y=7L} = 0, \\ \frac{\partial p}{\partial x}\bigg|_{x=L} = \frac{\partial p}{\partial y}\bigg|_{y=2L} = \frac{\partial p}{\partial x}\bigg|_{x=3L} = \frac{\partial p}{\partial y}\bigg|_{y=6L} = 0, \end{cases} \quad (37)$$

363 This indicates that Dirichlet boundary conditions with a value of zero are ap-
 364 plied on the outer edges, while Neumann boundary conditions with zero normal
 365 gradient are applied on the other edges. It can be demonstrated that the Poisson
 366 equation has the following analytical solution:

$$p(x, y) = \sin[\psi_x(x)] \sin[\psi_y(y)], \quad (38)$$

367 where the functions $\psi_x(x)$ and $\psi_y(y)$ are constructed as piecewise polynomials
 368 to satisfy the boundary conditions, i.e.,

$$\begin{aligned} \psi_x(x) &= x [A_1 + A_2(x - L) + A_3(x^2 - 4Lx + 3L^2)], \\ A_1 &= \frac{\pi}{L}, \quad A_2 = \frac{1}{3L} \left(\frac{\pi}{2L} - A_1 \right), \quad A_3 = \frac{1}{7L} \left[-\frac{\pi}{8L^2} - A_2 \right], \\ \psi_y(y) &= y [B_1 + B_2(y - 2L) + B_3(y^2 - 8Ly + 12L^2)], \\ B_1 &= \frac{\pi}{4L}, \quad B_2 = \frac{1}{6L} \left(\frac{\pi}{4L} - B_1 \right), \quad B_3 = \frac{1}{7L} \left[-\frac{\pi}{4L^2} - B_2 \right]. \end{aligned} \quad (39)$$

369 The righthand term $f(x, y)$ is defined by the Laplacian of the analytical
 370 solution $p(x, y)$.

371 On a uniform grid, the spatial step size is set as $\Delta x = \Delta y = L/k_n$, where k_n
 372 is a given constant controlling the number of grid points. By comparing with
 373 the analytical solution, we evaluated the spatial accuracy order of the algorithm.

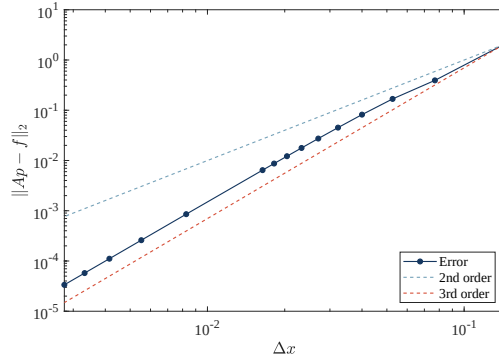


Figure 3: Precision order of cross-shape geometry.

374 As illustrated in Figure 3, due to the second-order central difference scheme,
 375 the solver achieves the expected second-order rate of convergence, confirming
 376 that the proposed domain decomposition algorithm correctly reproduces the
 377 precision of the underlying difference scheme.

378 3.2. GMRES iteration

379 As introduced in Section 2, the cross-flow field in Figure 2 is decomposed
 380 into five rectangular subregions. As introduced in Section 2, the cross-flow field
 381 in Figure 2 is decomposed into five rectangular subregions. The solution of the
 382 central subdomain Ω_2 is achieved by solving the Schur complement system via
 383 the proposed preconditioned GMRES iteration. To demonstrate the benefits
 384 of the proposed FFT-based preconditioner in enhancing iterative performance,
 385 we solve the sub-equation on Ω_2 using different preconditioners. We consider
 386 two subspace dimensions m and two equation sizes k_n and evaluate the residual
 387 reduction during the iterative process. The results are presented in Figure
 388 4. To demonstrate the benefits of the proposed FFT-based preconditioner in
 389 enhancing iterative performance, we solve the sub-equation on Ω_2 using different
 390 preconditioners. We consider two subspace dimensions m and two equation sizes
 391 k_n and evaluate the residual reduction during the iterative process. The results
 392 are presented in Figure 4.

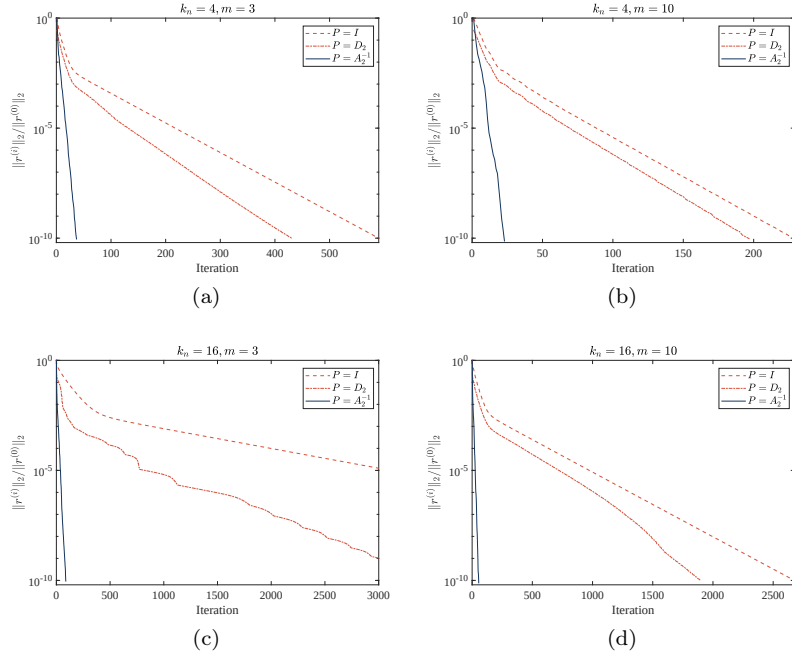


Figure 4: Comparison among the proposed preconditioner and classic preconditioners with different dimensions of subspaces and equation sizes

393 In the comparison presented in Figure 4, we evaluate the acceleration in
 394 convergence achieved by two preconditioning operators. The case $P = I$ indi-
 395 cates that no preconditioner is applied, and the sub-equation with the Schur
 396 complement, $(A_2 - S_2)\mathbf{p}_2 = \mathbf{f}'_2$, is solved directly. $P = D_2$ represents the use of
 397 the Jacobi preconditioner, a widely adopted classical preconditioner. Its prin-
 398 ciple involves using the inverse of the matrix D_2 , formed by the main diagonal
 399 elements of $(A_2 - S_2)$, as the preconditioning operator. $P = A_2^{-1}$ represents
 400 the proposed preconditioning operator as described in Section 2.4. As can be
 401 observed, the proposed preconditioner not only guarantees convergence for all
 402 equation sizes tested but also exhibits a significantly higher convergence rate
 403 than the two methods used for comparison. The number of iterations required
 404 to reach a given tolerance is reduced by a factor of more than seven. This
 405 reduction in iteration count is fundamental to the efficiency of the proposed
 406 algorithm. By employing a low-complexity computation for each iteration, the
 407 proposed algorithm achieves a notable reduction in the total number of itera-
 408 tions.

409 There exists a relationship between grid resolution and the optimal GM-
 410 RES subspace dimension for efficient convergence. To assess the impact of grid
 411 number and subspace dimension on the algorithm, we evaluated the effects of
 412 different grid numbers and subspace dimensions on the convergence of GMRES
 413 iterations, as shown in Figure 5.

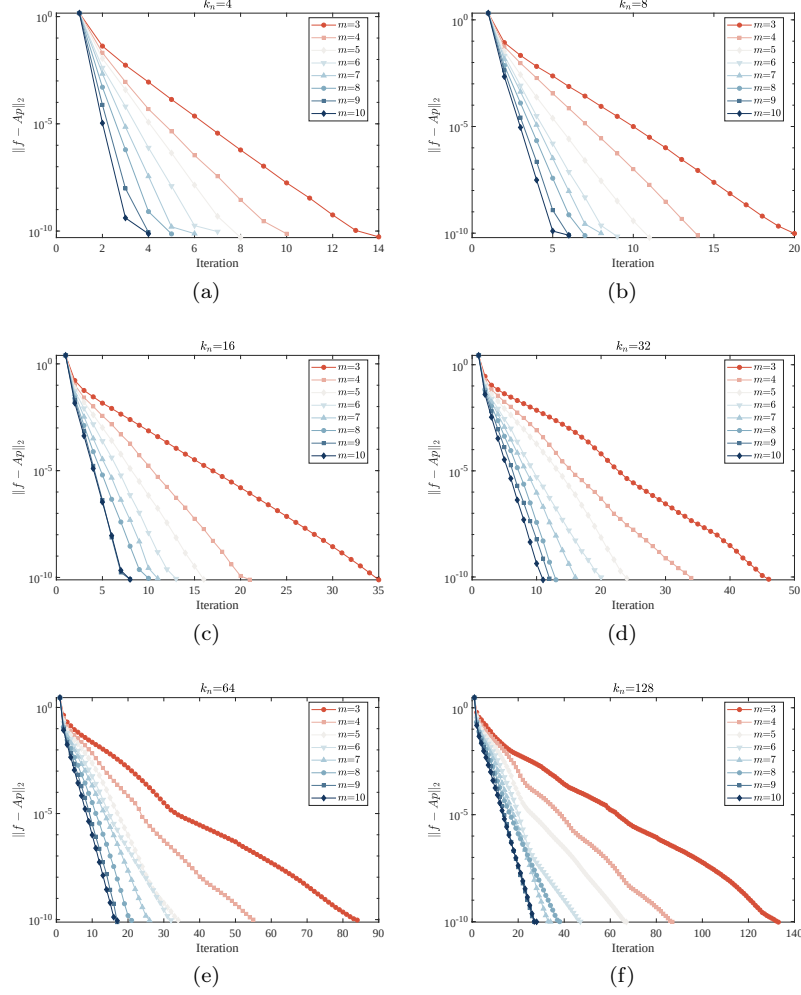


Figure 5: The iterative residual reduction in GMRES iteration with different size of the equation k_n and different dimension of subspace of GMRES m .

As shown in Figure 5, the iteration count increases with k_n (matrix size), a trend similar to that observed in other GMRES-like algorithms. It is well known that the FFT on simple geometries offers a direct solution without iterations; therefore, its computational cost remains consistent regardless of the matrix size. Given that the total computational cost of an iterative algorithm is determined by both the number of iterations and the computational cost per iteration, we next evaluate the impact of matrix size on the iteration count, and the results are presented in Figure 6.

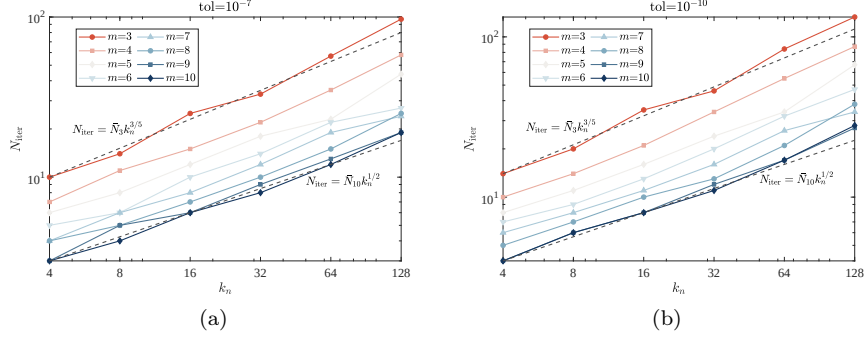


Figure 6: The iteration number for convergence under different residual tolerances.

Figure 6 illustrates the iteration counts for two representative residual convergence criteria, 10^{-7} and 10^{-10} . Two power-law fitting curves, depicted as dashed lines in Figure 6, are introduced as references of computational complexity order. Note that $\bar{N}_3$ and $\bar{N}_{10}$ are fitting constants independent of k_n . As can be seen, regardless of the subspace dimension m , the iteration count scales approximately with $k_n^{1/2}$. Since the computational complexity of each iteration is $O(k_n^2 \log(k_n))$, the estimated total computational complexity is $O(k_n^{5/2} \log(k_n))$. This represents a significant improvement compared to the complexity of traditional methods, which is typically higher than $O(k_n^3)$. The actual computation time of the algorithm is further assessed below to demonstrate its computational efficiency.

3.3. Computational efficiency

The results in Figure 5 indicate that the subspace dimension m has a significant impact on the algorithm, specifically in three aspects: 1. When the subspace dimension m is small, the number of iterations increases, inevitably slowing down the computational efficiency. 2. When the subspace dimension is large, the time for each GMRES iteration increases. An excessively large subspace dimension m can even lead to an increase in the computational cost of solving the coefficient equation within the subspace itself. Therefore, to evaluate the impact of the subspace dimension m on computational efficiency, we assessed the influence of the subspace dimension m on the GMRES computation time. The results are shown in Figure 7.

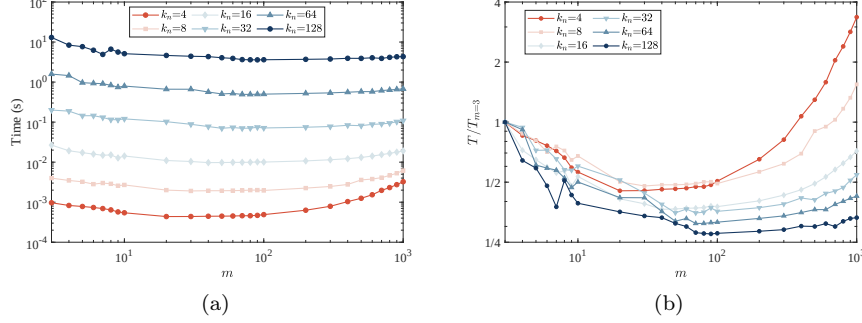


Figure 7: Computational time with different subspace dimension m and equation size k_n .

In Figure 7(a), the impact of the subspace dimension m on the total GMRES computation time is presented for six different equation sizes, denoted by k_n . It can be observed that for each matrix size, the influence of the subspace dimension m on the total GMRES time aligns with theoretical expectations; specifically, both excessively small and excessively large subspace dimensions m lead to increased computation time. To more clearly illustrate the effect of the subspace dimension m on computation time, Figure 7(b) provides a comparison of computation times at various subspace dimensions m against a baseline. This baseline is defined as the GMRES computation time for each equation size k_n when $m = 3$. It is clearly evident that when the subspace dimension m is approximately less than 80, the computation time decreases as m increases. Conversely, when the subspace dimension m exceeds 100, the computation time rapidly increases with increasing m . Consequently, for all parallel efficiency tests, $m = 80$ was consistently chosen as the optimal subspace dimension.

To validate the performance of the proposed algorithm, we conduct two numerical experiments: First, we compare the total solution time of our DDM-based solver against a direct solve of the global system for the cross-shaped domain. Since the complex geometry precludes a direct application of FFT-based methods to the global problem, we employ a sparse GMRES solver from the Eigen C++ library [30] as a benchmark. This comparison, presented in Figure 8(a), is designed to demonstrate the overall effectiveness of the DDM strategy. Second, to specifically evaluate the efficiency of our fast sub-domain solver, we compare its performance against Eigen's sparse GMRES solver on a single rectangular sub-domain (taken as Ω_2). This test isolates the performance of the GMRES solver component from the overhead of the DDM framework. The results of this direct solver comparison are shown in Figure 8(b).

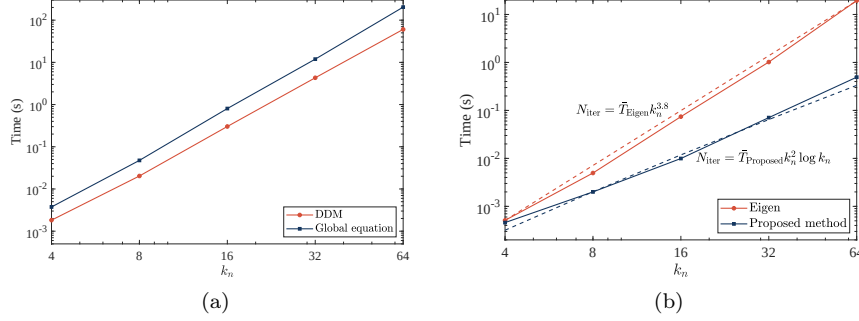


Figure 8: Computational time comparison for the cross-shaped domain. (a) Total solution time for the DDM solver vs. a direct global solve, both based on Eigen. (b) Solution time for a single sub-domain using the FFT-based GMRES solver (Proposed method) vs. Eigen solver.

In this comparison, all solvers were evaluated under identical operating environments, encompassing both hardware and parallel computing settings. Additionally, the Eigen solver employed sparse matrix storage and computation techniques. As shown in Figure 8(a), for any given equation size, the computational efficiency of the domain decomposition-based algorithm is several times greater than that of directly solving the global system of equations. This demonstrates that the adoption of a domain decomposition strategy significantly enhances computational efficiency compared to directly solving the global system using GMRES.

Furthermore, Figure 8(b) illustrates that the proposed algorithm exhibits a notable computational efficiency advantage over Eigen for large-scale problems. This superiority stems from differences in computational complexity, which aligns with our prior analysis. To quantify this performance advantage, the solution times plotted in Figure 8(b) are fitted to curves (dashed lines) that represent the observed computational scaling, where $\bar{T}_{\text{Eigen}}$ and $\bar{T}_{\text{Proposed}}$ are fitting constants. A comparison reveals that the computation time for traditional GMRES and similar solvers, when evaluated with respect to k_n , scales approximately as $k_n^{3.8}$. In contrast, the solution time for the algorithm proposed in this study scales approximately as $k_n^2 \log k_n$. Both trends closely match their respective theoretical estimations. While $k_n = 64$, which corresponds to a total of approximately 1.4×10^5 grid points, the proposed algorithm achieves a 40-times speedup compared to the Eigen-based solver.

Furthermore, due to fundamental limitations inherent in the computational principles of Krylov subspace methods, optimizing complexity cannot be achieved merely by replacing preconditioners without altering the underlying per-iteration algorithm or other core algorithmic components. Consequently, the GMRES algorithm equipped with an FFT-based preconditioner, as proposed in this study, offers substantial advantages for computations involving large matrices.

498 4. Conclusion

499 In this study, we propose a GMRES solution algorithm employing an FFT-
500 based preconditioner for solving the Poisson equation in complex geometries.
501 This algorithm achieves an $O(N \log N)$ complexity for Poisson equation solu-
502 tions within complex domains, demonstrating significant computational advan-
503 tages for problems involving large mesh sizes. The specific contributions of this
504 work are as follows:

- 505 1. A matrix equation-based FFT-type algorithm framework was constructed.
506 Unlike traditional FFT-based Poisson equation solvers that often employ
507 spectral space expansion strategies, this framework presents an accelerated
508 FFT-based solution algorithm leveraging eigenvalue transformations.
- 509 2. Building upon the matrix equation-based FFT-type algorithm framework,
510 the integration of FFT algorithms with DDMs was first realized. This
511 extends the applicability of FFT-type Poisson equation solvers to complex
512 geometries formed by concatenating multiple regions.
- 513 3. A GMRES algorithm with an FFT-based preconditioner was proposed.
514 Compared to traditional preconditioners, this significantly reduces the
515 number of GMRES iterations for sub-equations. Furthermore, an assess-
516 ment of the GMRES iteration count indicates that the complexity of the
517 FFT-based Poisson equation solver is only increased to $O(k_n^{5/2} \log k_n)$,
518 which still presents a significant advantage over traditional algorithms
519 with $O(k_n^3)$ computational complexity, such as the standard GMRES al-
520 gorithm.
- 521 4. An in-house C++ parallel solver was developed based on FFTW. Its com-
522 putational efficiency was compared with the Eigen solver on a parallel
523 platform. The results demonstrate that the developed algorithm exhibits
524 an $O(k_n^2 \log k_n)$ computational complexity, which is significantly superior
525 to the $O(k_n^4)$ computational complexity of traditional GMRES algorithms.
526 For GMRES solutions of matrices with a scale of $2^{15} \times 2^{15}$, the algorithm
527 proposed in this study achieves a computational efficiency improvement
528 of more than 20 times.

529 Acknowledgement

530 This work was supported by Guangdong Science and Technology Fund,
531 Grant 2022B1515120009. No external datasets, either public or private, were
532 utilized in this study; all computational results were generated solely through
533 the methods described. The in-house solver developed and mentioned in this
534 research has been open-sourced and is publicly available at https://github.com/ffskibkwi/Focalors_Poisson. All computational test results presented
535 in this paper are fully reproducible using the algorithms detailed herein.

536 During the preparation of this work the authors used Gemini in order to
537 improve the English language, correct grammar and punctuation, and enhance
538 the clarity and conciseness of the text, without generating any scientific content
539

or ideas. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the published article.

References

- [1] J. L. Jodra, I. Gurrutxaga, J. Muguerza, A. Yera, Solving poisson’s equation using fft in a gpu cluster, *Journal of Parallel and Distributed Computing* 102 (2017) 28–36.
- [2] J. W. Cooley, J. W. Tukey, An algorithm for the machine calculation of complex fourier series, *Mathematics of computation* 19 (90) (1965) 297–301.
- [3] G. Skölleremo, A fourier method for the numerical solution of poisson’s equation, *Mathematics of Computation* 29 (131) (1975) 697–711.
- [4] J. Xie, J. He, Y. Bao, X. Chen, A low-communication-overhead parallel dns method for the 3d incompressible wall turbulence, *International Journal of Computational Fluid Dynamics* 35 (6) (2021) 413–432.
- [5] J.-C. He, Y. Bao, X. Chen, Turbulent boundary layers in thermal convection at moderately high rayleigh numbers, *Physics of Fluids* 36 (2) (2024).
- [6] B. Han, J. Sim, Convergent sixth-order compact finite difference method for variable-coefficient elliptic pdes in curved domains, *Journal of Scientific Computing* 104 (2) (2025) 1–46.
- [7] A. Coco, G. Russo, High-order finite-difference ghost-point methods for elliptic problems in domains with curved boundaries, *Open Mathematics* 22 (1) (2024) 20240072.
- [8] W. Liu, G. Xu, X. Cheng, Y. Wang, A sixth-order finite difference model coupled with the matched interface and boundary method for underwater acoustic propagation simulations, *Journal of Computational Physics* 524 (2025) 113694.
- [9] A. B. Aithal, A. Ferrante, A fast pressure-correction method for incompressible flows over curved walls, *Journal of Computational Physics* 421 (2020) 109693.
- [10] N. Nikitin, Finite-difference method for incompressible navier–stokes equations in arbitrary orthogonal curvilinear coordinates, *Journal of Computational Physics* 217 (2) (2006) 759–781.
- [11] P. Costa, A fft-accelerated multi-block finite-difference solver for massively parallel simulations of incompressible flows, *Computer Physics Communications* 271 (2022) 108194.
- [12] Y. Wang, F. Hou, W. Wang, Fft-based efficient poisson solver in nonrectangular domain, *Computer Animation and Virtual Worlds* 34 (3-4) (2023) e2185.

- 577 [13] C. S. Peskin, Flow patterns around heart valves: a numerical method,
578 *Journal of computational physics* 10 (2) (1972) 252–271.
- 579 [14] A. Gilmanov, F. Sotiropoulos, A hybrid cartesian/immersed boundary
580 method for simulating flows with 3d, geometrically complex, moving bod-
581 ies, *Journal of computational physics* 207 (2) (2005) 457–492.
- 582 [15] L. A. V. d. Albuquerque, M. F. d. S. Villela, F. P. Mariano, Numerical sim-
583 ulation of flows using the fourier pseudospectral method and the immersed
584 boundary method, *Axioms* 13 (4) (2024) 228.
- 585 [16] X. Chen, B. Yin, F. Dong, Synergistic effect of ribs and particles on flow
586 and heat transfer in channels using the cfd-dem-ibm method, *Powder Tech-
587 nology* 431 (2024) 119082.
- 588 [17] Z. Chen, C. S. Peskin, A fourier spectral immersed boundary method
589 with exact translation invariance, improved boundary resolution, and a
590 divergence-free velocity field, *Journal of Computational Physics* 509 (2024)
591 113048.
- 592 [18] M. Uhlmann, An immersed boundary method with direct forcing for the
593 simulation of particulate flows, *Journal of computational physics* 209 (2)
594 (2005) 448–476.
- 595 [19] J. Lian, Q. Yao, Z. Jiang, An overlapping ibm-piso algorithm with an fft-
596 based poisson solver for parallel incompressible flow simulations, *Fluids*
597 10 (7) (2025) 176.
- 598 [20] Z. Zhao, H. Dong, W. Ying, Kernel free boundary integral method for 3d in-
599 compressible flow and linear elasticity equations on irregular domains, *Com-
600 puter Methods in Applied Mechanics and Engineering* 414 (2023) 116163.
- 601 [21] Y. Ren, H. Feng, S. Zhao, A fft accelerated high order finite difference
602 method for elliptic boundary value problems over irregular domains, *Jour-
603 nal of Computational Physics* 448 (2022) 110762.
- 604 [22] Y. Ren, S. Zhao, A fft accelerated fourth order finite difference method for
605 solving three-dimensional elliptic interface problems, *Journal of Computa-
606 tional Physics* 477 (2023) 111924.
- 607 [23] C. Li, Y. Ren, G. Long, E. Boerman, S. Zhao, A fast sine transform ac-
608 celerated high-order finite difference method for parabolic problems over
609 irregular domains, *Journal of Scientific Computing* 95 (2) (2023) 49.
- 610 [24] Z. Song, S.-K. Lai, B. Wu, A new mib-based time integration method
611 for transient heat conduction analysis of discrete and continuous systems,
612 *International Journal of Heat and Mass Transfer* 222 (2024) 125153.
- 613 [25] R. Verzicco, Immersed boundary methods: Historical perspective and fu-
614 ture outlook, *Annual Review of Fluid Mechanics* 55 (1) (2023) 129–155.

- 615 [26] Y. Saad, M. H. Schultz, Gmres: A generalized minimal residual algorithm
616 for solving nonsymmetric linear systems, SIAM Journal on scientific and
617 statistical computing 7 (3) (1986) 856–869.
- 618 [27] J. Schur, Über potenzreihen, die im innern des einheitskreises beschränkt
619 sind. (1917).
- 620 [28] M. Frigo, S. G. Johnson, The design and implementation of FFTW3, Pro-
621 ceedings of the IEEE 93 (2) (2005) 216–231, special issue on “Program
622 Generation, Optimization, and Platform Adaptation”.
- 623 [29] S. G. Johnson, M. Frigo, A modified split-radix FFT with fewer arithmetic
624 operations, IEEE Trans. Signal Processing 55 (1) (2007) 111–119.
- 625 [30] G. Guennebaud, B. Jacob, et al., Eigen v3, <http://eigen.tuxfamily.org>
626 (2010).